

Architekturbericht: Transition wissenschaftlicher Python-Ökosysteme zur Referenzarchitektur (GenesisAeon 2026)

Die Transformation eines explorativen wissenschaftlichen Forschungslabors – in diesem Kontext das Repository "unified-mandala" – in eine stabile, öffentlich nutzbare Referenzarchitektur erfordert fundamentale strukturelle Neuausrichtungen. Das geplante GenesisAeon-Ökosystem, bestehend aus einem gehärteten Kern-Monorepo und 48 unabhängig versionierten Microservice-Paketen aus diversen wissenschaftlichen Domänen, stellt singuläre Anforderungen an Dependency-Isolierung, Reproduzierbarkeit und deterministische Release-Zyklen. Die Integration eines obligatorischen "Diamond Interface" zur Gewährleistung von Reproduzierbarkeit und Zenodo-Archivierung erfordert zudem eine strikte, protokollbasierte Entkopplung. Die nachfolgende tiefgreifende Analyse evaluiert den modernsten Stand der Technik (2025/2026) für große Python-Monorepos, Plugin-Architekturen, Governance-Modelle und Performance-Kopplung, um eine datengetriebene Grundlage für die anstehenden Architekturentscheidungen zu liefern.

1. Python-Monorepo-Tooling (Stand 2026)

Die Verwaltung komplexer Python-Monorepos hat zwischen 2024 und 2026 einen vollständigen Paradigmenwechsel durchlaufen. Die klassische Debatte zwischen Monorepo und Polyrepo gilt durch die Evolution moderner Tooling-Infrastruktur als weitgehend gelöst, da die historischen Nachteile von Monorepos – wie mangelnde Isolierung und langsame CI-Pipelines – durch spezialisierte Werkzeuge eliminiert wurden¹. Für ein Ökosystem mit 48+ Subpaketen ist die Wahl des primären Build- und Workspace-Tools die kritischste initiale Entscheidung.

Der aktuelle De-facto-Standard für die Verwaltung von Python-Workspaces ist das in Rust geschriebene Tool `uv` von Astral. Die Einführung von `uv workspaces`, konzeptionell stark inspiriert durch Rusts Paketmanager `Cargo`, ermöglicht es, mehrere unabhängige Python-Pakete in einem einzigen Repository zu verwalten². Die überlegene Architektur von `uv` zeigt sich in der lokalen Isolierung: Wird in einem Sub-Paket-Verzeichnis der Befehl `uv sync` ausgeführt, generiert das Tool in Sekundenbruchteilen ein virtuelles Environment, das ausschließlich die explizit in der lokalen `pyproject.toml` deklarierten Abhängigkeiten enthält¹. Dies schließt andere Workspace-Pakete ein, die direkt aus dem Quellcode verlinkt werden, verhindert jedoch das unabsichtliche Importieren von Modulen, die nicht als direkte Abhängigkeit definiert sind¹. Dieses deterministische Verhalten eliminiert das Problem der "Mystery Dependencies", welches ältere Pip-basierte Workflows dominierte. Die Performance

von uv ist dabei um Größenordnungen höher als bei klassischen Tools².

Im direkten Vergleich offenbaren alternative Build-Systeme spezifische Limitationen für das GenesisAeon-Szenario. Das von Google entwickelte Bazel skaliert theoretisch unbegrenzt und erzwingt hermetische Builds, verlangt jedoch einen massiven administrativen Overhead durch die Pflege kryptischer BUILD-Files (Starlark) und ist primär für polyglotte Megarepos von Enterprise-Konzernen konzipiert⁵. Pants (in der v2.x-Architektur) bietet eine exzellente automatisierte Dependency-Inferenz für Python, wird jedoch aufgrund seiner Komplexität zunehmend durch die simplere Architektur von uv verdrängt⁶. Tools wie Nx und Turborepo sind historisch stark auf das JavaScript/TypeScript-Ökosystem fokussiert und erfordern für Python beträchtliche Workarounds⁶. Hatch und PDM bieten zwar grundlegende Workspace-Unterstützungen, erreichen jedoch weder die Ausführungsgeschwindigkeit noch die strenge deterministische Isolationsarchitektur von uv⁸.

In der Praxis wird diese moderne Architektur bereits in massiven Open-Source-Projekten produktiv eingesetzt. Apache Airflow, eines der größten Python-Monorepos weltweit mit über 1,2 Millionen Zeilen Code, verwaltet über 120 separate Provider-Pakete über uv workspaces¹. Anstatt als monolithischer Block zu agieren, sind die 120 Pakete durch das Workspace-Konzept logisch und physisch getrennt. Ein weiteres architektonisches Muster, das sich 2026 etabliert hat, ist die "Polyolith"-Architektur, welche Komponenten als austauschbare Bausteine ("Bricks") in einem Monorepo organisiert und native Unterstützung für uv bietet¹⁰. Um die Problematik extrem langsamer Pre-Commit-Hooks in solchen Monorepos zu lösen, setzen Projekte wie Airflow und CPython auf prek, einen in Rust geschriebenen Drop-in-Ersatz für pre-commit, der Workspace-aware ist und Hooks für unabhängige Projekte parallelisiert ausführt⁴.

Die konkrete Trennung von Kern und optionalen Erweiterungen wird in modernen uv-Workspaces über eine strikte Ordnerstruktur abgebildet. In der Wurzel des Repositories definiert eine primäre pyproject.toml im Block [tool.uv.workspace] die Mitglieder (z. B. members = ["core", "plugins/*"]). Jedes der 48 Plugins besitzt ein eigenes Verzeichnis mit eigener pyproject.toml. Die Plugins deklarieren ihre Abhängigkeit zum Kern explizit über core = { workspace = true }³. Dies ersetzt fehleranfällige und schwer wartbare Git-Submodule vollständig und erlaubt es dem System, den Kern und ausgewählte Plugins dynamisch zu linken, ohne auf externe Paketquellen (PyPI) zugreifen zu müssen¹.

Monorepo-Tool	Primärer Architektur-Fokus	Lernkurve / Komplexität	Hermetische Dependency-Isolierung	Eignung für GenesisAeon
uv workspaces	Nativ Python (Rust-basiert)	Niedrig	Exzellent (strikte venv-Trennung)	Sehr hoch. Optimale Balance aus Performance, Einfachheit und nativer

				Isolation.
Pants (v2)	Polyglott, Fokus auf Python	Hoch	Sehr gut (Dependency-Inferenz)	Mittel. Hohe Präzision, aber signifikanter Konfigurations- Overhead.
Bazel	Massives Polyglott (Google)	Sehr hoch	Exzellent (Sandboxing)	Niedrig. Zu massiv für ein reines Python-Ökosy- stem mit 48 Paketen.
Hatch / PDM	Python-Standa- rd-Projekte	Niedrig	Befriedigend	Niedrig. Funktional unterlegen gegenüber der Performance von uv.
Nx / Turborepo	JavaScript / TypeScript	Mittel	Schwach (für Python)	Sehr niedrig. Erfordert inoffizielle Plugins und Workarounds.

2. Plugin-Architektur-Patterns für wissenschaftliche Python-Pakete

Die Architektur eines Plugin-Systems diktiert maßgeblich die Skalierbarkeit und Wartbarkeit eines Software-Ökosystems. Für GenesisAeon, wo 48 Domänen-spezifische Microservices dynamisch auf einen Kern aufsetzen, muss die Kopplung extrem lose gestaltet sein. Der Kern darf zur Laufzeit und Kompilierzeit keine Kenntnis über die konkrete Existenz oder Implementierung der Plugins besitzen.

Im Jahr 2026 existieren klare, ausgereifte Best Practices für die Entkopplung. Die Standardbibliothek bietet ab Python 3.10 mit `importlib.metadata.entry_points()` eine hochperformante und robuste Lösung für das "Plugin-Discovery", welche die veraltete

pkg_resources-Bibliothek vollständig abgelöst hat¹⁵. Dieser Mechanismus iteriert über installierte Pakete, die sich in ihrer pyproject.toml unter einer definierten Gruppe (z. B. genesisaeon.plugins) registriert haben. Für komplexe, ereignisgesteuerte Systeme wird häufig pluggy (die Engine hinter dem pytest-Framework) eingesetzt¹⁷. pluggy orchestriert Plugins über ein formelles Hook-System (@hookspec, @hookimpl), etabliert eine HookRelay-Architektur und erlaubt Vorher-/Nachher-Interventionen bei Methodenaufrufen¹⁹. Im Gegensatz dazu stehen schwere Frameworks wie OpenStack Stevedore, die stark an spezifische Infrastrukturen gebunden sind und in modernen, leichtgewichtigen Architekturkonzepten kaum noch Verwendung finden²⁰.

Um die vollständige Entkopplung zu garantieren, setzen moderne Architekturen auf strukturierte Typisierung mittels typing.Protocol (PEP 544), anstatt traditionelle abstrakte Basisklassen (ABCs) zu erzwingen. Der GenesisAeon-Kern definiert das "Diamond Interface" lediglich als Protokoll. Ein Plugin importiert dieses Protokoll nicht zur Laufzeit aus dem Kern, sondern implementiert lediglich Methoden mit exakt den geforderten Signaturen²¹. Der Kern wiederum nutzt importlib.metadata, um die Entry Points zu laden, und validiert die geladene Klasse optional über isinstance(plugin, DiamondInterface) (sofern das Protokoll mit @runtime_checkable dekoriert ist). Dieses Muster verhindert zirkuläre Importe und ermöglicht es, Plugins als völlig autarke Repositories oder Microservices zu entwickeln, die den Kern nicht als harte Abhängigkeit benötigen²³.

Das Management von Inkompatibilitäten über Versionsgrenzen hinweg (Capability-Negotiation) wird in reifen Projekten über Manifest-Dictionaries und statische API-Versionierung gelöst. Ein exemplarisches Muster zeigt das wissenschaftliche Paket SpectroChemPy²⁴. Plugins exponieren dort eine Methode (z. B. plugin_info()), die ein Metadaten-Dictionary inklusive einer plugin_api_version zurückgibt²⁴. Der Kern lädt den Entry Point initial nicht in den Ausführungskontext, sondern inspiziert zunächst diese API-Version. Entspricht die Major-Version des Plugins nicht der des Kerns, wird das Plugin mit einer präzisen PluginVersionError-Ausnahme abgewiesen, ohne dass inkompatibler Code ausgeführt wird²⁴. Dies schützt das "Diamond Interface" vor Laufzeitfehlern bei Versionsinkonsistenzen.

Plugin-Muster / Tool	Architektur-Prinzip	Discovery-Mechanismus	Stärken für GenesisAeon
importlib.metadata + PEP 544	Strukturelle Typisierung	Entry Points (Standardbibliothek)	Maximale Entkopplung, keine externen Abhängigkeiten, hochperformant.
pluggy	Hook-basiert (Eventing)	Entry Points / Manuelle Registry	Ideal, wenn Plugins den Programmfluss des Kerns aktiv

			modifizieren müssen.
Stevedore (OpenStack)	Objektorientierte Registry	Entry Points	Veraltet und zu schwergewichtig für ein reines, modulares Scientific-Setup.
Subclassing (ABCs)	Nominale Typisierung	Direct Imports / Metaclasses	Hohe Kopplung, zwingt Plugins, den Kern als harte Runtime-Abhängigkeit zu laden.

3. Reproduzierbarkeit & Determinismus im wissenschaftlichen Kern

Die Krise der Reproduzierbarkeit in der Wissenschaft verlangt von Referenzarchitekturen absolute Determinismus-Garantien. Ein 2026 veröffentlichtes Modell muss auch Jahre später bit-identisch ausgeführt werden können, unbeeinflusst von zwischenzeitlichen Updates im PyPI-Ökosystem oder der Abschaltung von Servern.

Für die Fixierung der Abhängigkeiten hat sich ein dualer Konsens gebildet. Innerhalb reiner Python-Pipelines ist `uv.lock` der unumstrittene Standard². Es generiert deterministische, plattformübergreifende kryptografische Hashes für den gesamten Abhängigkeitsbaum und lässt sich in Sekunden auflösen. Für das übergeordnete Ziel der "Offline-Reproduzierbarkeit als Hartkriterium" reicht ein Python-Lockfile jedoch nicht aus, da es C-Compiler, Systembibliotheken (wie HDF5 oder NetCDF) und Betriebssystem-Abhängigkeiten ignoriert. Hier ist das Nix-Ökosystem, speziell *Nix Flakes*, der Goldstandard für wissenschaftliche Umgebungen⁹. Nix deklariert die gesamte Systemumgebung puristisch und funktional; der Bauprozess erfolgt in einer hermetisch abgeriegelten Sandbox ohne Netzwerkzugriff. Für GenesisAeon empfiehlt sich die Symbiose: `uv` für die Entwickler-Ergonomie und schnelle Iteration, gekapselt in einer `flake.nix` für die finale Zenodo-Archivierung, um bit-genaue Reproduzierbarkeit des gesamten Ausführungskontextes zu garantieren⁹.

Zur Validierung der numerischen und physikalischen Modelle ist das "Property-based Testing" (PBT) dem klassischen "Example-based Testing" überlegen. Das Framework Hypothesis dominiert das Python-Ökosystem weiterhin²⁵. Entwickler schreiben keine statischen Tests (z.B. `assert func(2) == 4`), sondern definieren Invarianten (z.B. Energieerhaltung), während Hypothesis den Eingaberaum stochastisch nach Randfällen durchsucht, die das Modell brechen²⁵. Neuere Entwicklungen (Stand 2025/2026) umfassen den Einsatz von LLM-Agenten,

die aus Docstrings und Typ-Annotationen automatisiert hochkomplexe Hypothesis-Strategien generieren, sowie alternative Frameworks wie Kea2, die speziell auf die Skalierung von PBT in großen Applikationen abzielen²⁵. Benchmarks zur Evaluierung von PBT-Tools (wie PBT-Bench) belegen die Reife dieser Methodik für komplexe Tensoren und Zustandsmaschinen²⁵.

Deterministische Benchmark-Suiten in reifen Projekten erfordern die vollständige Eliminierung von Live-APIs und stochastischer Varianz. Reale Validierungsdaten werden nicht zur Laufzeit heruntergeladen, sondern durch Tools wie pooch (intensiv im SciPy-Ökosystem genutzt) versioniert gecacht oder direkt via Git LFS in das Repository eingebunden. Um die Ausführung von Simulationen deterministisch zu halten, werden Pseudozufallszahlengeneratoren (PRNGs) global über Fixtures (z.B. pytest-randomly) mit fixierten Seeds initialisiert.

Netzwerkschnittstellen werden über Mocks (wie responses oder VCR.py) auf lokal aufgezeichnete Kassetten umgeleitet, um sicherzustellen, dass die Tests im hermetischen Offline-Modus der Nix-Sandbox nicht fehlschlagen.

4. Versionierungs- und Release-Automatisierung über viele Pakete

Das historisch gewachsene Problem inkonsistenter Versionsstände und manuell verwalteter Releases über 48 Pakete hinweg führt in Monorepos zwangsläufig zu administrativen Engpässen. Die Automatisierung dieser Prozesse ist 2026 ein gelöstes Problem, erfordert jedoch die strikte Einhaltung von Konventionen.

Für Python-Monorepos hat sich das Tool python-semantic-release (PSR) seit der Einführung des dedizierten conventional-monorepo-Parsers in Version 10.4.0 (Ende 2025) als robusteste Lösung etabliert²⁸. PSR liest die Git-Historie und analysiert Commit-Nachrichten, die dem "Conventional Commits"-Standard folgen. In einem Monorepo werden Commits auf spezifische Pakete gescopt (z. B. feat(climate): add non-linear solver oder fix(core): resolve memory leak)³¹. Über die Definition von path_filters in den jeweiligen pyproject.toml-Dateien der 48 Plugins wird PSR angewiesen, ausschließlich Commits zu bewerten, die Dateien im jeweiligen Plugin-Verzeichnis modifizieren³¹. Push-Ereignisse auf den Main-Branch triggern GitHub Actions, die PSR ausführen: Das Tool kalkuliert den semantischen Versionssprung für jedes betroffene Paket isoliert, generiert ein dediziertes Changelog, aktualisiert die Versionsvariablen, erzeugt einen Git-Tag (z. B. climate-v2.1.0) und publiziert das Wheel auf PyPI³¹.

Alternativen wie Changesets (adaptiert für Python)³³ verlangen, dass Entwickler neben dem Code eine Markdown-Datei ("Intent") ablegen, was zwar die Qualität der Changelogs erhöht, aber einen manuellen Prozessschritt erzwingt³³. Googles release-please bietet ebenfalls starken Support für Monorepos, ist jedoch stark an das GitHub-Ökosystem gebunden und weniger tief in reine Python/uv-Pipelines integriert³⁴. PSR in Kombination mit uv ist die präferierte Wahl, da es den Build- und Release-Prozess vollständig nativ hält³¹. Ein weiteres Tool für Changelogs ist towncrier, das News-Fragmente in einem changelog.d-Verzeichnis sammelt und bei einem Release assembliert, seit Version 23.10 auch mit offiziellem Monorepo-Support³⁵. Für eine vollautomatisierte CI/CD-Pipeline bietet PSR jedoch die

überlegenere End-to-End-Automatisierung.

Hinsichtlich der Versionsstrategie empfiehlt die Literatur für ein System aus einem Kern und 48 lose gekoppelten domänenspezifischen Plugins die *Independent Versioning*-Strategie¹.

"Fixed/Lockstep Versioning" (wie es etwa bei Angular oder Babel praktiziert wird) würde bedeuten, dass eine Änderung im Klima-Plugin künstliche Major-Bumps in den Physik- und Netzwerk-Plugins erzwingt. Independent Versioning bewahrt die Autonomie der wissenschaftlichen Module¹.

Die Architektur des "Diamond Interface" wird in der Systemarchitektur als "Interface Stability Contract" oder "Protocol Versioning" bezeichnet. Um Chaos zu vermeiden, wird das Interface in das Kern-Repository integriert, welches einen strikt konservativen Lebenszyklus erhält.

Änderungen am Diamond Interface erfordern zwingend einen Major-Release des Kerns. Die 48 Plugins deklarieren in ihren Abhängigkeiten eine Pinning-Strategie (z. B. `genesis-core >= 2.0, < 3.0`). So wird garantiert, dass ein Plugin, das für das Interface v2 geschrieben wurde, niemals mit dem Interface v3 ausgeführt wird.

Release-Tool	Ansatz	Eignung für Python-Monorepos	Automatisierungsgrad
python-semantic-release	Analyse von Conventional Commits	Exzellent (ab v10.4.0)	Sehr hoch. Vollautomatischer Versions-Bump und PyPI-Release.
Changesets	Manuelle Intent-Dateien per PR	Sehr gut	Hoch. Erfordert Disziplin bei der Erstellung der Markdown-Dateien.
release-please	Commit-Analyse (Google)	Gut	Hoch. Sehr GitHub-zentriert.
towncrier	News-Fragment-Dateien	Mittel	Mittel. Generiert nur Changelogs, übernimmt nicht den automatischen Version-Bump.

5. Architecture Decision Records (ADRs) — aktuelle

Tooling-Landschaft

Der Übergang von einem explorativen Labor zu einer Referenzarchitektur ist von massiven technologischen Weichenstellungen geprägt (z. B. die Wahl von uv oder die Definition des Diamond Interface). Um diese Rationale für zukünftige Entwickler und Forscher nachvollziehbar zu archivieren, sind Architecture Decision Records (ADRs) obligatorisch. Sie verhindern das "blinde Reversieren" einmal getroffener, gut durchdachter Architekturentscheidungen³⁸.

Die Tooling-Landschaft hat sich auf das MADR-Format (Markdown Architectural Decision Records) als Standard verständigt³⁸. MADR ist eine leichte Evolution des ursprünglichen ADR-Formats, bietet optionale Detail-Sektionen und ist maschinenlesbar, was den Overhead für Entwickler minimiert³⁸. Während einfache Repositories ADRs oft manuell als Markdown-Dateien pflegen, hat sich für professionelle Referenzarchitekturen das Tool log4brains als Best Practice herauskristallisiert³⁸. Im Gegensatz zu den älteren, bash-basierten adr-tools bietet log4brains eine CLI zur Erstellung, automatischen Nummerierung (z. B. ADR-004-use-uv.md) und Generierung einer durchsuchbaren statischen Website (Knowledge Base), die nahtlos in CI-Pipelines integriert werden kann³⁸.

In aktiven Open-Source-Projekten werden ADRs üblicherweise im Verzeichnis docs/decisions/ oder docs/architecture/adr/ abgelegt⁴¹. Eine Best Practice für die Verlinkung von ADRs zum Quellcode ist die ausschließliche Nutzung von GitHub-Tree-Permalinks (mit spezifischen Git-Hashes), um zu verhindern, dass Code-Referenzen durch spätere Refactorings ins Leere laufen ("Dead Links"). Der Status eines ADR (Proposed, Accepted, Deprecated, Superseded) wird im YAML-Frontmatter der Markdown-Datei gepflegt, was Tools wie log4brains befähigt, veraltete Entscheidungen visuell auf der generierten Projekt-Website zu markieren⁴⁰.

ADR-Tool	Format	Funktionen	Zielgruppe / Projektgröße
log4brains	MADR	CLI, Auto-Nummerierung, Statische Website-Generierung	Mittel bis Groß. Ideal für Referenzarchitekturen und verteilte Teams.
Manuelles MADR	MADR	Keine Automatisierung	Klein. Startups, kleine Labore.
adr-tools	Klassisch	CLI-basiert, rein textuell	Veraltet. Wird zunehmend durch log4brains ersetzt.

PlantUML / Structurizr	C4 / UML	Diagram-as-Code	Ergänzend zu ADRs für visuelle Systemarchitektur.
---------------------------	----------	-----------------	---

6. Governance, Lizenzierung & Förderfähigkeit für wissenschaftliche Software

Die Professionalisierung von "Research Software Engineering" (RSE) hat dazu geführt, dass Förderorganisationen ihre Kriterien bis 2026 stark formalisiert und objektivierbar gemacht haben. Um förderfähig zu sein, reicht es nicht mehr, Code auf GitHub zu publizieren; die Architektur muss strukturelle Compliance-Vorgaben erfüllen.

Der deutsche **Sovereign Tech Fund (STF)**, eine maßgebliche Institution für die Finanzierung kritischer Open-Source-Infrastruktur, legt bei der Evaluation von Projekten strikte Metriken an. Zu den Kriterien gehören "Prevalence" (die messbare Nutzung des Projekts als Abhängigkeit in anderen Technologien), "Relevance" (Sektorenübergreifende Bedeutung) und "Vulnerability" (Risiko durch technische Schulden oder Maintainer-Burnout)⁴². Die Grundvoraussetzung für eine STF-Förderung ist die Nutzung einer von der Open Source Initiative (OSI) anerkannten Lizenz, explizit ohne "Non-Commercial"-Einschränkungen⁴³.

Auf EU- und globaler Ebene (z. B. EU-Horizon, NSF) sind die **FAIR4RS-Prinzipien** (Findable, Accessible, Interoperable, Reusable for Research Software) zur normativen Pflicht geworden⁴⁴. Die Erfüllung dieser Prinzipien erfordert persistente Maschinenlesbarkeit. Ein obligatorisches Instrument zur Umsetzung der "Findability" ist die Implementierung der Datei CITATION.cff (Citation File Format) im Root-Verzeichnis des Repositories⁴⁶. Diese Datei stellt sicher, dass Zitationsdatenbanken und Plattformen wie Zenodo die Metadaten des Codes (Autoren, DOI, Versionen) automatisiert extrahieren können⁴⁸.

Hinsichtlich der Lizenz-Compliance fordern moderne Regulierungen (z.B. im Rahmen des EU Cyber Resilience Act) zunehmend die Einhaltung der **REUSE-Spezifikation**⁴⁵. Dies erfordert, dass jede einzelne Quellcodedatei im Header mit einem eindeutigen SPDX-License-Identifier (z. B. SPDX-License-Identifier: Apache-2.0) sowie dem Copyright-Inhaber versehen wird. Diese Praxis erlaubt automatisierte Lizenz-Audits und garantiert Rechtssicherheit bei der Weiternutzung von Teilmodulen⁴⁹.

Die Lizenzstrategie für ein System aus Kern und 48 Plugins erfordert eine bewusste Trennung. Die Literatur und Analyse vergleichbarer Ökosysteme empfiehlt dringend eine einheitliche, permissive Lizenz (wie Apache 2.0, MIT oder BSD-3-Clause) für den Kern, um kommerzielle und akademische Adaption hürdenfrei zu ermöglichen. Da die 48 Plugins jedoch über das "Diamond Interface" und importlib dynamisch zur Laufzeit geladen werden und nicht statisch gegen den Kern gelinkt sind, können für spezifische Plugins gemischte Lizenzmodelle toleriert werden (z. B. ein GPL-lizenziertes Plugin für spezielle numerische Algorithmen), ohne dass das gesamte Monorepo von der Viralität der GPL infiziert wird. Diese lose Kopplung isoliert rechtliche Risiken²¹.

7. Mehrsprachigkeit / Performance-kritische Module

Obwohl das GenesisAeon-Ökosystem primär in Python entwickelt wird, erfordern rechenintensive Domänen (wie PDE-Solver in der Physik, Tensorberechnungen oder die Traversierung großer semantischer Netzwerke) den Einsatz von kompilierten Systemsprachen. Die Schnittstellentechnologien für diese Hybridarchitekturen haben sich massiv optimiert. Für C++-basierte Architekturen ist nanobind im Jahr 2026 der uneingeschränkte Nachfolger des traditionellen pybind11. Es generiert Binärdateien, die bis zu 5x kleiner sind, reduziert die Kompilierzeit um das Vierfache und weist einen bis zu 10x geringeren Runtime-Overhead beim Überqueren der Python/C++-Grenze auf⁵⁰. Projekte mit existierender C++-Legacy nutzen nanobind aufgrund dieser signifikanten Effizienzsteigerungen standardmäßig⁵³.

Für Neuentwicklungen hat sich jedoch die Kombination **Python + Rust (über PyO3 und maturin)** als modernste und sicherste Architektur durchgesetzt⁴¹. Rust bietet Memory-Safety ohne Garbage Collection, verhindert Data Races auf Compiler-Ebene und fügt sich nahtlos in Toolchains ein, die auf uv (ebenfalls in Rust geschrieben) basieren. Konferenzen für Hochleistungs-Python-Tools fokussieren sich zunehmend auf PyO3 als präferierte Schnittstelle zur Überwindung von Performance-Flaschenhälsen⁵⁴.

Die stark beworbene Programmiersprache **Mojo** zielt zwar explizit auf wissenschaftliches Rechnen ab und verspricht C-Performance bei Python-Syntax, erreicht jedoch 2026 noch nicht die notwendige Reife für eine referenzfähige RSE-Architektur⁵⁵. Defizite bei der vollständigen Open-Source-Verfügbarkeit der Toolchain, Unklarheiten in der Standardbibliothek und die starke Bindung an das Modular-Ökosystem machen Mojo für ein unabhängiges, offline-reproduzierbares Projekt derzeit zu einem unkalkulierbaren Risiko⁵⁵. Die Python+Rust-Allianz bleibt der pragmatische Standard⁵⁶.

Das Hartkriterium der absoluten Offline-Reproduzierbarkeit erfordert jedoch, dass GenesisAeon auch ohne komplexe C++/Rust-Build-Chains lauffähig bleibt. Hierfür wird das "Pure Python Fallback"-Pattern empfohlen: Kompilierte Erweiterungen werden als optionale Abhängigkeiten verpackt. Der Code versucht, die performanten Module zu importieren (try: import genesis_rust_accel); schlägt dies mit einem ImportError fehl, greift das System transparent auf eine reine Python-, NumPy- oder JAX-Implementierung zurück. JAX bietet hierbei durch seinen Just-in-Time-Compiler (XLA) exzellente Performance für reine Vektoroperationen, ohne dedizierte C-Erweiterungen pflegen zu müssen. Zudem verspricht das in Python 3.14 (2026) offiziell eingeführte Free-Threading (Aufhebung des Global Interpreter Lock, GIL) und der experimentelle JIT-Compiler signifikante native Performance-Steigerungen⁵⁸.

Technologie	Primäre Sprache	Performance-Gewinn	Eignung für GenesisAeon
PyO3 + maturin	Rust	Sehr hoch	Exzellent. Memory-safe,

			moderner Standard, harmoniert perfekt mit uv-Infrastruktur.
nanobind	C++	Sehr hoch	Sehr gut. Der direkte Nachfolger von pybind11, ideal bei C++-Legacy.
Mojo	Mojo	Hoch (theoretisch)	Niedrig. Fehlende infrastrukturelle Reife und unzureichende Open-Source-Autonomie (Stand 2026).
JAX / Numba	Python (JIT)	Hoch (für Mathe/Tensoren)	Sehr gut als Fallback-Lösung zur Wahrung der Pure-Python-Kompatibilität.

8. Vergleichsfälle: "Labor → Referenzarchitektur"-Übergänge realer Projekte

Der methodische Übergang von experimentellem Forschungscode (wie unified-mandala) zu stabilen Referenzarchitekturen ist eine bekannte Hürde in der wissenschaftlichen Informatik. Die Analyse historischer Transitionen liefert Blaupausen für GenesisAeon.

PyTorch (Merge mit Caffe2): PyTorch begann 2016 als stark forschungsfokussiertes Framework mit dynamischen Berechnungsgraphen, stieß jedoch an Grenzen bei der produktiven Skalierung. Die Transition zur Referenzarchitektur erfolgte 2018 durch den massiven architektonischen Merge mit *Caffe2*, Facebooks auf Produktion getrimmtem Framework⁶⁰. Dieser Schritt kombinierte die intuitive Frontend-Erfahrung von PyTorch mit dem hochoptimierten C++-Backend von Caffe2⁶². Begleitet wurde der Cut von Planungsdokumenten wie der ONNX-Spezifikation zur Entkopplung von Frameworks⁶⁰. Schließlich wurde der Code aus dem Meta-Monopol gelöst und der Linux Foundation als "PyTorch Foundation" übergeben, um Governance-Ziele zu erfüllen⁶⁰. Das ursprüngliche "Labor" wurde dabei durch eine strikte Gewaltenteilung von Core (ATen C++ Backend) und

domain-spezifischen Bibliotheken (wie TorchVision) abgelöst⁶¹.

Astropy (Das Core + Affiliates Modell): Das Astropy-Projekt löste 2011 exakt das Problem, vor dem GenesisAeon steht: Eine Vielzahl unkoordinierter, unabhängiger Astronomie-Pakete in Python⁶⁵. Die architektonische Entscheidung war der Aufbau eines monolithischen "Core"-Pakets für essenzielle Funktionalitäten (Koordinaten, FITS-I/O) und die Etablierung eines streng kuratierten "Affiliated Packages"-Ökosystems⁶⁵. Diese Architektur gleicht dem Ansatz "1 Kern + 48 Plugins" auf frappierende Weise. Ein Vorgänger-Repo im klassischen Sinne gab es nicht; Astropy formierte sich als koordinierte Konsolidierung. Affilierte Pakete wurden nicht einfach integriert, sondern mussten rigorose Codequalitäts-, Test- und Dokumentations-Standards erfüllen und das Astropy-Core-Interface bedienen, um in die offizielle Registry aufgenommen zu werden⁶⁵. Der Schnitt wurde durch ein Koordinationskomitee vorgenommen, das den klassischen BDFL-Ansatz (Benevolent Dictator For Life) bewusst mied⁶⁵. Das Astropy-Projekt bietet bis heute die sauberste strukturelle Vorlage für GenesisAeon.

SciPy-Ökosystem: SciPy durchläuft keinen punktuellen "Cut", sondern eine kontinuierliche Transition von Labor-Code (oft Legacy-Fortran/C) hin zu einer stabilen API. Die architektonische Trennung erfolgt hier durch formelle Planungsdokumente, die NEPs (NumPy Enhancement Proposals), welche exakt dem ADR-Konzept entsprechen. Funktionalitäten werden zuerst als explorative Submodule in `scipy.sandbox` getestet und erst bei Stabilität in den Kern überführt.

9. Semantische Graphen / Abhängigkeits-Kartierung über große Codebasen

Um den amorphen Labor-Code (unified-mandala) strukturiert in einen Kern und 48 Plugins zu zerlegen, reichen einfache Textanalysen oder Tools wie `pipdeptree` nicht aus, da diese lediglich deklarierte Meta-Abhängigkeiten anzeigen. Die Analyse muss auf Ebene des Abstrakten Syntaxbaums (AST) erfolgen, um die tatsächliche semantische Kopplung des Codes zu messen. Der aktuelle Standard zur Durchsetzung und Visualisierung von Architektur-Grenzen ist das Tool `import-linter`⁶⁸. Es analysiert den Code auf AST-Ebene mittels der Bibliotheken `grimp` und `NetworkX` und baut einen vollständigen, gerichteten Import-Graphen auf⁷⁰. Durch die Definition von Verträgen in einer `layers.yml` oder `.importlinter` Konfiguration können Architekturregeln formell durchgesetzt werden⁶⁸. Für GenesisAeon bedeutet dies: Man definiert den Core als niedrigste Schicht ("Layer") und die 48 Plugins als höhere Schichten. `import-linter` schlägt in der CI-Pipeline fehl, sobald ein Modul aus dem Klimawissenschafts-Plugin fälschlicherweise das Neuro-Plugin importiert oder wenn der Core auf ein Plugin zugreift, was eine zyklische Quer-Abhängigkeit und einen Verstoß gegen das "Diamond Interface"-Muster bedeuten würde⁷⁰.

Neuere Alternativen auf dem Markt sind `tach`, das Dependency-Visualisierung mit Enforcement kombiniert⁷², sowie `ArchUnitPython`, welches das Konzept des Java-Klassikers `ArchUnit` auf Python portiert und Architekturtests direkt in der `pytest`-Suite ermöglicht⁷⁴. Diese Tools machen die "Technical Debt" durch Kopplungsmetriken messbar und verhindern, dass die

saubere Referenzarchitektur im Laufe der Zeit durch unsaubere Importe wieder in einen Monolithen erodiert⁷².

Für die automatisierte Identifikation von Modulen, die den neuen "Core" bilden sollen, bietet die Graphentheorie etablierte Methoden. Werden AST-basierte Abhängigkeitsgraphen in NetworkX geladen, lassen sich Zentralitätsmaße wie "Betweenness Centrality" oder "Eigenvector Centrality" berechnen⁷⁰. Module mit extrem hohem "Fan-in" (viele andere Module importieren sie) und niedrigem "Fan-out" (sie selbst importieren kaum fremden Code) weisen einen hohen Grad an Kohäsion auf und sind die mathematisch belegbaren Kandidaten für die Extraktion in das Kern-Repository⁷². Diese algorithmische Herangehensweise eliminiert emotionale oder rein historische Begründungen beim Schneiden der Microservices.

Analyse-Tool	Ansatz	Fokus	Eignung für das Refactoring
import-linter	AST-basierter Graph (grimp)	Durchsetzung von Schichtenarchitektur (Contracts)	Sehr hoch. Ideal zur Verhinderung von Architektur-Erosion in CI.
ArchUnitPython	Unit-Tests für Architektur	Tests via pytest	Hoch. Fügt sich nahtlos in bestehende Test-Infrastruktur ein.
tach	Rust-basiertes CLI	Visualisierung & Enforcement	Mittel. Sehr modern, aber import-linter ist etablierter.
NetworkX (Custom)	Graphenanalyse	Fan-in/Fan-out Berechnung	Sehr hoch. Zwingend erforderlich für die initiale Core-Kandidaten-Findung.

Fazit: Priorisierte Kurzliste für GenesisAeon

Auf Basis der umfassenden Analyse der Tooling-Landschaft 2026 kristallisieren sich klare Sieger für das definierte Anforderungsprofil heraus.

Wenn heute nur fünf fundamentale Tool-Entscheidungen für das GenesisAeon-Projekt (Wissenschaftliches Python-Ökosystem, 1 Kern + 48 lose gekoppelte Plugins, Offline-Reproduzierbarkeit als Hartkriterium, Förderfähigkeit als Ziel) getroffen werden müssten, sind dies die robustesten Optionen:

1. **Monorepo & Environment Management:** uv workspaces (Astral).
 - *Rationale:* Löst die Dependency-Auflösung in Sekundenbruchteilen, erzwingt die notwendige physische Isolierung (venv-Ebene) der 48 Plugins und bietet durch deterministische uv.lock-Files die Basis für Reproduzierbarkeit¹.
2. **Plugin-Discovery & Interface-Architektur:** importlib.metadata kombiniert mit PEP 544 (typing.Protocol).
 - *Rationale:* Maximale Entkopplung ohne jeglichen Framework-Overhead. Der Kern definiert das "Diamond Interface" lediglich als strukturelles Protokoll. importlib identifiziert und lädt die Plugins zur Laufzeit über Entry Points aus den 48 Paketen¹⁵.
3. **Versionierung & Release-Automatisierung:** python-semantic-release (PSR v10.4.0+).
 - *Rationale:* Der neu eingeführte conventional-monorepo-Parser befähigt das System, die 48 Plugins völlig autark im Monorepo zu versionieren. Gesteuert durch einfache Commit-Prefixes (feat(climate): ...) und path_filters wird der administrative Release-Overhead vollständig eliminiert²⁸.
4. **Architektur-Enforcement in CI:** import-linter.
 - *Rationale:* Ein unverzichtbares Werkzeug, um sicherzustellen, dass die 48 Plugins ausschließlich gegen den Core programmieren. Das Tool bricht den Build-Prozess ab, sobald Querabhängigkeiten zwischen Plugins entstehen, und garantiert so den dauerhaften Erhalt der Referenzarchitektur⁶⁸.
5. **Governance & Compliance (Förderfähigkeit):** log4brains (für ADRs) + REUSE / CITATION.cff.
 - *Rationale:* Garantiert die Förderfähigkeit durch Organisationen wie den Sovereign Tech Fund (STF) oder die EU. Die statisch generierte ADR-Wissensdatenbank von log4brains schafft institutionelles Vertrauen, während die CITATION.cff und REUSE-Header die wissenschaftliche FAIR4RS-Compliance sicherstellen⁴⁰.

Referenzen

1. Episode #540 - Modern Python monorepo with uv and prek | Talk Python To Me Podcast,
<https://talkpython.fm/episodes/show/540/modern-python-monorepo-with-uv-and-prek>
2. Scaling a Python Org: Fast Builds, Consistent Environments, Happy Engineers | Doppel,
<https://www.doppel.com/blog/scaling-python-org-fast-builds-consistent-environments-happy-engineers>
3. Using workspaces | uv - Astral Docs,
<https://docs.astral.sh/uv/concepts/projects/workspaces/>
4. FOSDEM Talk: Modern Python Monorepo with uv, Workspaces, and prek,

- <https://pydevtools.com/blog/fosdem-talk-modern-python-monorepo/>
5. Top 5 Monorepo Tools for 2025 | Best Dev Workflow Tools - Aviator,
<https://www.aviator.co/blog/monorepo-tools/>
 6. Best Monorepo Build Tools for Engineering Teams (2026) | Sourcegraph,
<https://sourcegraph.com/blog/monorepo-build-tools>
 7. Monorepos in 2026: Turborepo vs Nx vs Bazel — What Actually Works - DEV Community,
<https://dev.to/zny10289/monorepos-in-2026-turborepo-vs-nx-vs-bazel-what-actually-works-3c7d>
 8. Workspaces and monorepos | PyCharm Documentation - JetBrains,
<https://www.jetbrains.com/help/pycharm/python-workspaces.html>
 9. Python dependency management is a dumpster fire - Niels Cautaerts,
<https://nielscautaerts.xyz/python-dependency-management-is-a-dumpster-fire.html>
 10. Polyolith: a Monorepo Architecture : r/Python - Reddit,
https://www.reddit.com/r/Python/comments/1m9so08/polyolith_a_monorepo_architecture/
 11. The Python tools for the Polyolith Architecture: a Monorepo Architecture : r/madeinpython - Reddit,
https://www.reddit.com/r/madeinpython/comments/1m9aipa/the_python_tools_for_the_polyolith_architecture_a/
 12. GitHub - j178/prek: A fast Git hook manager written in Rust, designed as a drop-in alternative to pre-commit, reimaged., <https://github.com/j178/prek>
 13. prek - prek, <https://prek.j178.dev/>
 14. Workspaces and monorepo support (add sync --all-packages) · Issue #6935 · astral-sh/uv, <https://github.com/astral-sh/uv/issues/6935>
 15. importlib.metadata – Accessing package metadata — Python 3.14.6 documentation, <https://docs.python.org/3/library/importlib.metadata.html>
 16. importlib-metadata - PyPI, <https://pypi.org/project/importlib-metadata/>
 17. Changelog — pluggy 0.1.dev96+gfd08ab5 documentation - Read the Docs,
<https://pluggy.readthedocs.io/en/stable/changelog.html>
 18. Plugins case study: Pluggy - Eli Bendersky's website,
<https://eli.thegreenplace.net/2026/plugins-case-study-pluggy/>
 19. pluggy 0.1.dev108+gaa8d46145 documentation,
<https://pluggy.readthedocs.io/en/latest/>
 20. Comparison - Abilian Innovation Lab,
<https://lab.abilian.com/Tech/Python/Useful%20Libraries/Plugin%20Systems/Comparison/>
 21. How to Build Plugin Systems in Python - OneUptime,
<https://oneuptime.com/blog/post/2026-01-30-python-plugin-systems/view>
 22. Navigating Python's Dependency System: From Overload to Optimization | by Michael Balber | CyberArk Engineering | Medium,
<https://medium.com/cyberark-engineering/navigating-pythons-dependency-system-from-overload-to-optimization-6209cde2bbcd>
 23. Best practices for managing dynamic imports in plugin-based architecture? -

Python Help,

<https://discuss.python.org/t/best-practices-for-managing-dynamic-imports-in-plugin-based-architecture/99482>

24. Writing a Plugin — SpectroChemPy v0.9.3 0.9.3 documentation,
<https://www.spectrochempy.fr/0.9.3/devguide/plugins/architecture.html>
25. PBT-Bench: Benchmarking AI Agents on Property-Based Testing - arXiv,
<https://arxiv.org/html/2605.15229v2>
26. Finding bugs across the Python ecosystem with Claude and property-based testing - Anthropic Red, <https://red.anthropic.com/2026/property-based-testing/>
27. Kea2: Practical Property-based Testing for Mobile Apps - Ting Su,
<https://tingsu.github.io/files/fse26-Kea2.pdf>
28. CHANGELOG - python-semantic-release 10.5.3 documentation,
https://python-semantic-release.readthedocs.io/en/latest/misc/psr_changelog.html
29. Configuration - python-semantic-release 10.5.3 documentation,
<https://python-semantic-release.readthedocs.io/en/latest/configuration/configuration.html>
30. Releases · python-semantic-release/python-semantic-release - GitHub,
<https://github.com/rele kang/python-semantic-release/releases>
31. Releasing a Monorepo using uv Workspace and Python Semantic Release - Medium,
<https://medium.com/@asafshakarzy/releasing-a-monorepo-using-uv-workspace-and-python-semantic-release-0dafc889f4cc>
32. [bug] Monorepo `path\_filters` fails to reference paths outside the current package subtree · Issue #1380 - GitHub,
<https://github.com/python-semantic-release/python-semantic-release/issues/1380>
33. Intentional Releases: Why I Chose Changesets over Semantic-Release · Infra Bootstrap Tools,
<https://xnok.github.io/infra-bootstrap-tools/blog/intentional-releases-changesets/>
34. How to Implement Semantic Versioning Automation - OneUptime,
<https://oneuptime.com/blog/post/2026-01-25-semantic-versioning-automation/vi ew>
35. Daily Development Workflows — Backend.AI Documentation 26.4.4rc4 문서,
<https://docs.backend.ai/ko/latest/dev/daily-workflows.html>
36. How to Keep a Changelog in Markdown - Towncrier 25.8.0 documentation,
<https://towncrier.readthedocs.io/en/stable/markdown.html>
37. Release notes - Towncrier 23.11.0 documentation,
<https://towncrier.readthedocs.io/en/23.11.0/release-notes.html>
38. Demystifying Architectural Decision Records: Why Every Project Needs Them,
<https://www.dandoescode.com/blog/demystifying-architectural-decision-records-why-every-project-needs-them>
39. ADR Approaches: Comparison and Recommendations - calcopy,
<https://calcopy.kyleking.me/docs/adr-research/comparison-and-recommendation>

- [s/](#)
40. From Chaos to Clarity: The Essential Role of Architectural Decision Records (ADRs),
<https://adamcogan.com/2024/10/09/essential-role-of-architectural-decision-records/>
 41. meshmonk/docs/decisions/ADR-001-meshmonk-modernization.md,
<https://github.com/jsnyde0/meshmonk/blob/main/docs/decisions/ADR-001-meshmonk-modernization.md>
 42. Sovereign Tech Fund: how Germany funds critical open source | bartman.pl,
<https://www.bartman.pl/open-source/sovereign-tech-fund-how-germany-funds-critical-open-source/>
 43. Funding open source: case study on the Sovereign Tech Fund | Interoperable Europe Portal,
<https://interoperable-europe.ec.europa.eu/collection/open-source-observatory-osor/document/funding-open-source-case-study-sovereign-tech-fund>
 44. (PDF) FAIR AI models in high energy physics - ResearchGate,
https://www.researchgate.net/publication/376293323_FAIR_AI_Models_in_High_Energy_Physics
 45. The FAIR for Research Software Principles after two years: an adoption update,
<https://www.researchsoft.org/blog/2024-03/>
 46. tasks: Adopting FAIR research software practices | RSQKit - EVERSE project,
https://everse.software/RSQKit/fair_rs
 47. Awareness of FAIR and FAIR4RS among international research software funders - PMC,
<https://pmc.ncbi.nlm.nih.gov/articles/PMC12000615/>
 48. Applying FAIR4RS principles to develop an integrated modeling environment for the magnetic confinement fusion - PMC,
<https://pmc.ncbi.nlm.nih.gov/articles/PMC10485057/>
 49. The missing rung in the FOSS assurance ladder is cost sharing - Christoph Görn,
<https://xn--grn-sna.name/posts/foss-cost-sharing-standard-blog-2026-04-28/>
 50. nanobind Documentation,
https://nanobind.readthedocs.io/_/downloads/en/latest/pdf/
 51. pybind11 Documentation,
https://pybind11.readthedocs.io/_/downloads/en/stable/pdf/
 52. C++ API Reference (Core) - nanobind documentation,
https://nanobind.readthedocs.io/en/latest/api_core.html
 53. source/languages/python/index.rst - cta-computing - GitLab - CTAO,
<https://gitlab.cta-observatory.org/cta-computing/documentation/developer-documentation/-/blob/40ae5717d11a3407b6d9368e6d40954521c3dc1b/source/languages/python/index.rst>
 54. pretalx - EuroPython 2026,
<https://programme.europython.eu/europython-2026/schedule/>
 55. SPy and Mojo: first impressions vs. deeper reality - My SPy Book - Heptapod,
<https://fluiddyn.pages.heptapod.net/spy-book/versus-mojo>
 56. Programming Languages for Artificial Intelligence and Machine Learning: An Updated Analysis with Original Benchmarks and LLM Bia - Research Square,

- https://assets-eu.researchsquare.com/files/rs-9410893/v1_covered_d62ee578-0bbe-40dc-9e67-eacfef5ab776.pdf
57. Open question: what would you like to see from a Mojo package manager?,
<https://forum.modular.com/t/open-question-what-would-you-like-to-see-from-a-mojo-package-manager/2799>
 58. What's new in Python 3.14 — Python 3.14.6 documentation,
<https://docs.python.org/3/whatsnew/3.14.html>
 59. What's new in Python 3.14 — Python 3.14.6 ドキュメント - Python documentation,
<https://docs.python.org/ja/3/whatsnew/3.14.html>
 60. PyTorch - Wikipedia, <https://en.wikipedia.org/wiki/PyTorch>
 61. AlirezaShamsoshoara/PyTorchHistory: A comprehensive chronicle of PyTorch's evolution—from research tool to AI infrastructure powering modern deep learning, from 2001 to 2025 - GitHub,
<https://github.com/AlirezaShamsoshoara/PyTorchHistory>
 62. The Complete History and Evolution of PyTorch | Deep Learning Framework Timeline, <https://tensorgym.com/blog/pytorch-history>
 63. Ultimate guide to Caffe library in Python - Deepnote,
<https://deepnote.com/blog/ultimate-guide-to-caffe-library-in-python>
 64. Igniting Innovation: Evidence from PyTorch on Technology Control in Open Collaboration - Questrom World,
https://questromworld.bu.edu/platformstrategy/wp-content/uploads/sites/49/2025/07/PlatStrat2025_paper_145.pdf
 65. The Astropy Project: Building an inclusive, open-science project and status of the v2.0 software - ResearchGate,
https://www.researchgate.net/publication/322355094_The_Astropy_Project_Building_an_inclusive_open-science_project_and_status_of_the_v20_software
 66. (PDF) The Astropy Problem - ResearchGate,
https://www.researchgate.net/publication/309008609_The_Astropy_Problem
 67. (PDF) Astropy: A community Python package for astronomy - ResearchGate,
https://www.researchgate.net/publication/251566730_Astropy_A_community_Python_package_for_astronomy
 68. Import Linter, <https://import-linter.readthedocs.io/>
 69. layer-linter - PyPI, <https://pypi.org/project/layer-linter/>
 70. Linter for Python Architecture - Roman Imankulov,
<https://roman.pt/posts/python-architecture-linter/>
 71. Enforce import rules using the Python import linter,
<https://921kiyo.com/python-import-linter/>
 72. Python Code Quality Tools Beyond Linting - DEV Community,
<https://dev.to/lrscke/python-code-quality-tools-beyond-linting-42d8>
 73. Code Quality Tools Beyond Linting · astral-sh ruff · Discussion #20699 - GitHub,
<https://github.com/astral-sh/ruff/discussions/20699>
 74. ArchUnit but for Python: enforce architecture rules as unit tests. : r/django - Reddit,
https://www.reddit.com/r/django/comments/1tg3t62/archunit_but_for_python_enforce_architecture/

75. Building Modular Monoliths with Logical Boundaries Hexagonal Architecture and Internal Messaging - SoftwareSeni,
<https://www.softwareseni.com/building-modular-monoliths-with-logical-boundaries-hexagonal-architecture-and-internal-messaging/>
76. (PDF) Retrieval-Conditioned Topology Selection with Provable Budget Conservation for Multi-Agent Code Generation - ResearchGate,
https://www.researchgate.net/publication/404627927_Retrieval-Conditioned_Topology_Selection_with_Provable_Budget_Conservation_for_Multi-Agent_Code_Generation
77. Retrieval-Conditioned Topology Selection with Provable Budget Conservation for Multi-Agent Code Generation - arXiv, <https://arxiv.org/pdf/2605.05657>